

Lab Materials

- Build a simple sentiment analysis program with DSPy built-in modules.
- A custom agent with DSPy by building a “Name the Celebrity” game.



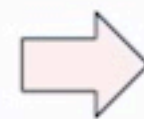
DSPy Backbone – Signature and Module

The backbone of DSPy programming consists of two classes:

- 1) `dspy.Signature`
- 2) `dspy.Module`.

- **dspy.Signature:** define the input/output contract of DSPy module

```
classify = dspy.Predict("sentence -> sentiment: int, language: str")
```



- **dspy.Module:** define the logic of interacting with LM. Takes user input and signature.

```
cot = dspy.ChainOfThought("question->answer")  
print(cot(question="what's the capital of France?"))
```


Signature

Signature (**dspy.Signature**) defines the input and output of LLM interaction.

Think of LLM as a function, signature defines the args and return value of the function.

- What I can give you (**dspy.InputField()**)
- What you should give me (**dspy.OutputField()**)
 - Type (**pydantic** format)
 - Extra information about the field via **`desc`** keyword

Two types of signatures:

- Class-based
- String-based

Class-based Signature

```
class Translation(dspy.Signature):  
    """Translate English sentence into Japanese."""  
  
    english_text: str = dspy.InputField()  
    japanese_text: str = dspy.OutputField()  
    hiragana: str = dspy.OutputField()  
    grammer_involved: list[str] = dspy.OutputField()  
    alternatives: list[tuple[str, str]] = dspy.OutputField(  
        desc="Alternatives and explanation"  
    )  
    difficulty: Literal["N1", "N2", "N3", "N4", "N5"] = dspy.OutputField(  
        desc="JLPT level"  
    )
```



Instruction



Extra information for the field



Field name



Type information



Input or output field

String-based Signature

```
classify = dsp.Predict("sentence -> sentiment: int, language: str")
```

Left side of "->" are the input fields, and
right side are the output fields,
separated by ","

Built-in Modules

 **Module (`dspy.Module`)** is the core unit of a DSPy program

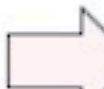
- Encapsulates logic for interacting with LMs

 **Simplest module: `dspy.Predict`**

- Formats user input
- Parses LM output based on a signature

 **Modules have configurable attributes**

- `demos`: Few-shot examples
- `lm`: Specifies which language model to use

  **Modules are composable and customizable**

- Can contain child modules
- Add custom logic for advanced workflows

Built-in Modules

Inside DSPy we provide a list of built-in modules to make it easy for users to get started:

- **dspy.Predict**: Simple LM interaction. The building block for complex modules
- **dspy.ChainOfThought**: ask LM to include reasoning along with the answer
- **dspy.ReAct**: the common abstraction of continuous tool calling
- **dspy.ProgramOfThought**: a special tool calling module, inside it the tool is code
- **dspy.Refine**: users can set a reward function + threshold, if threshold is unmet we issue retries.

Built-in Modules

```
cot = dsp.ChainOfThought("question->answer")  
print(cot(question="what's the capital of France?"))
```

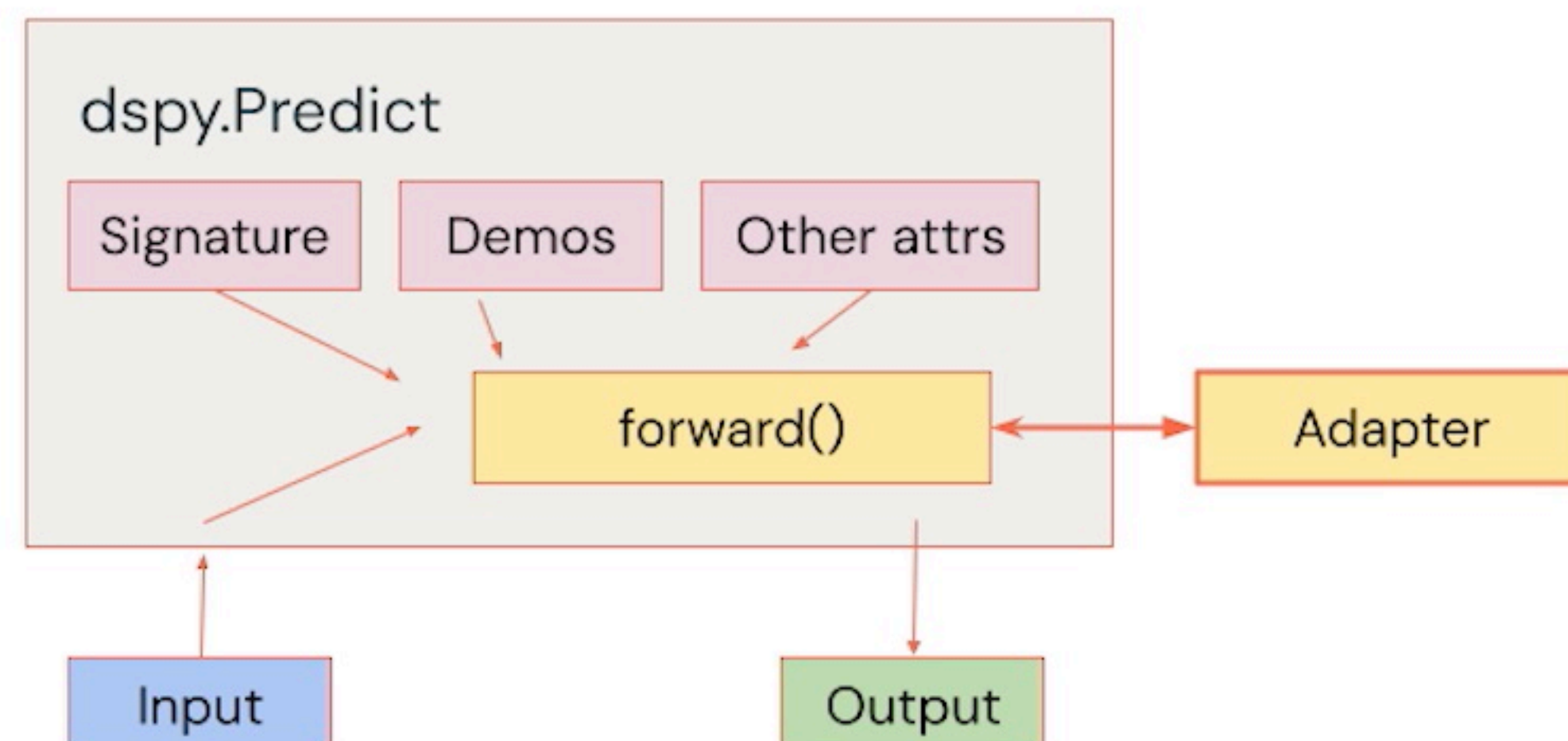
Built-in modules: <https://dsp.ai/api/modules/Module/>

Custom Modules

```
class MyProgram(dspy.Module):  
    def __init__(self, **kwargs):  
        super().__init__()  
  
        self.module1 = dspy.ChainOfThought(MySignature1)  
        self.module2 = dspy.ChainOfThought(MySignature2)  
  
    def forward(self, inputs, **kwargs):  
        x = preprocess(x)  
        x = self.module1(inputs)  
        x = midprocess(x)  
        x = self.module2(x)  
        return postprocess(x)
```

Define your forward logic in `forward` method, similar to PyTorch

Behind the Scene – Adapter



At a high level, `dspy.Predict` Composes a multi-turn prompt using:

- 📝 **User input** — formatted by the **signature** and **adapter**
- 🖋️ **Demos** — few-shot examples to guide the LM

Parses the LLM response

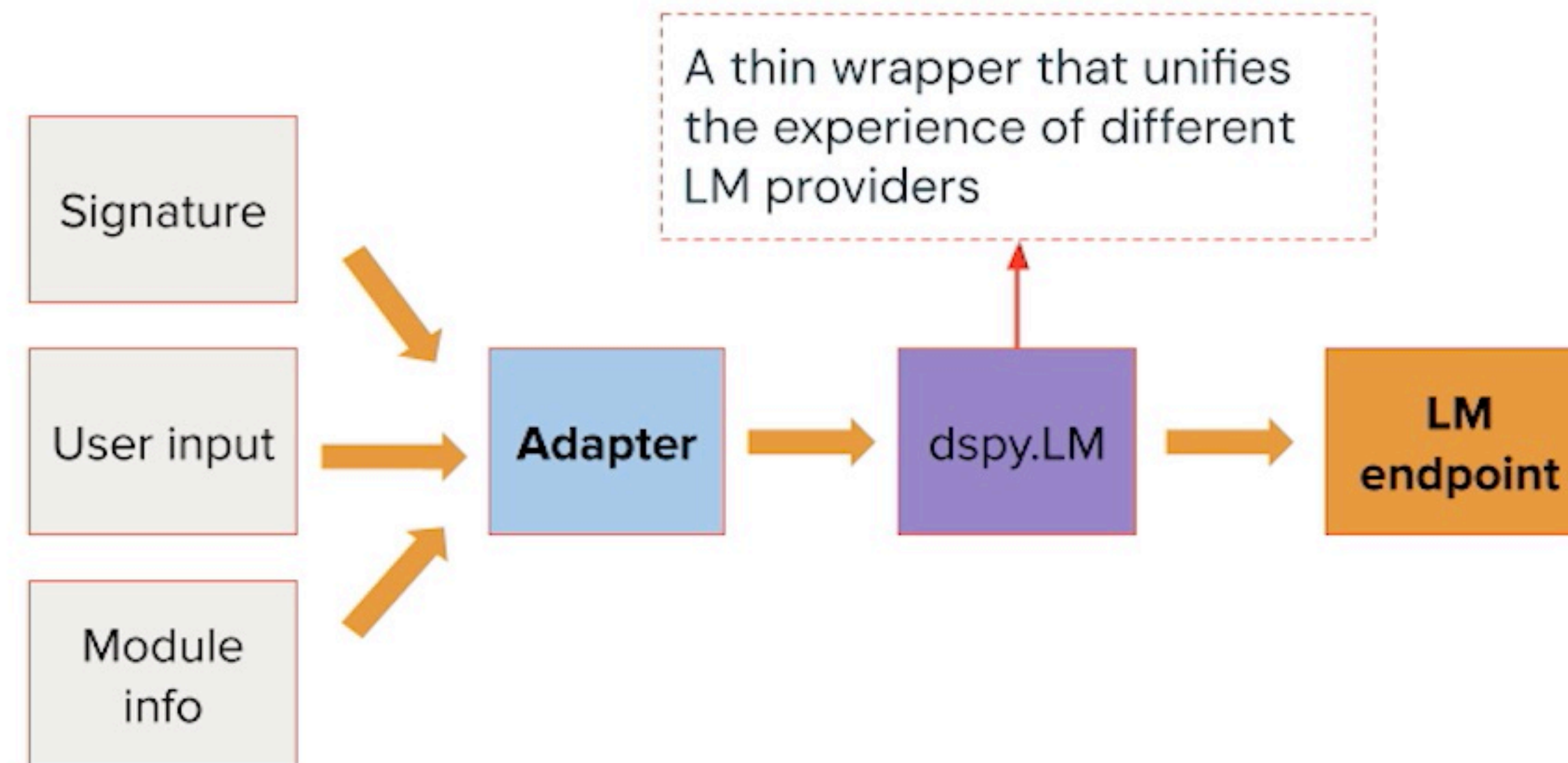
- Uses the same signature and adapter for structured output

Returns a `dspy.Prediction`

- A dict-like object containing the extracted output fields

Behind the Scene - Adapter (Input flow)

The adapter takes in signature, user input and module input (prebuilt few-shot examples), and transforms it into the familiar multi-turn messages with different roles.



Into the Adapter Input Flow

System message:

Your input fields are:

1. `question` (str)

Field Description

Source: Signature → fields name

Your output fields are:

1. `reasoning` (str)

2. `answer` (str)

All interactions will be structured in the following way, with the appropriate values filled in.

[[## question ##]]

{question}

Input/output format description

Source: Signature → fields name + info

[[## reasoning ##]]

{reasoning}

[[## answer ##]]

{answer}

[[## completed ##]]

In adhering to this structure, your objective is:

Given the fields `question`, produce the fields `answer`.

User message:

User message formatted as required

Source: Signature, user input

[[## question ##]]

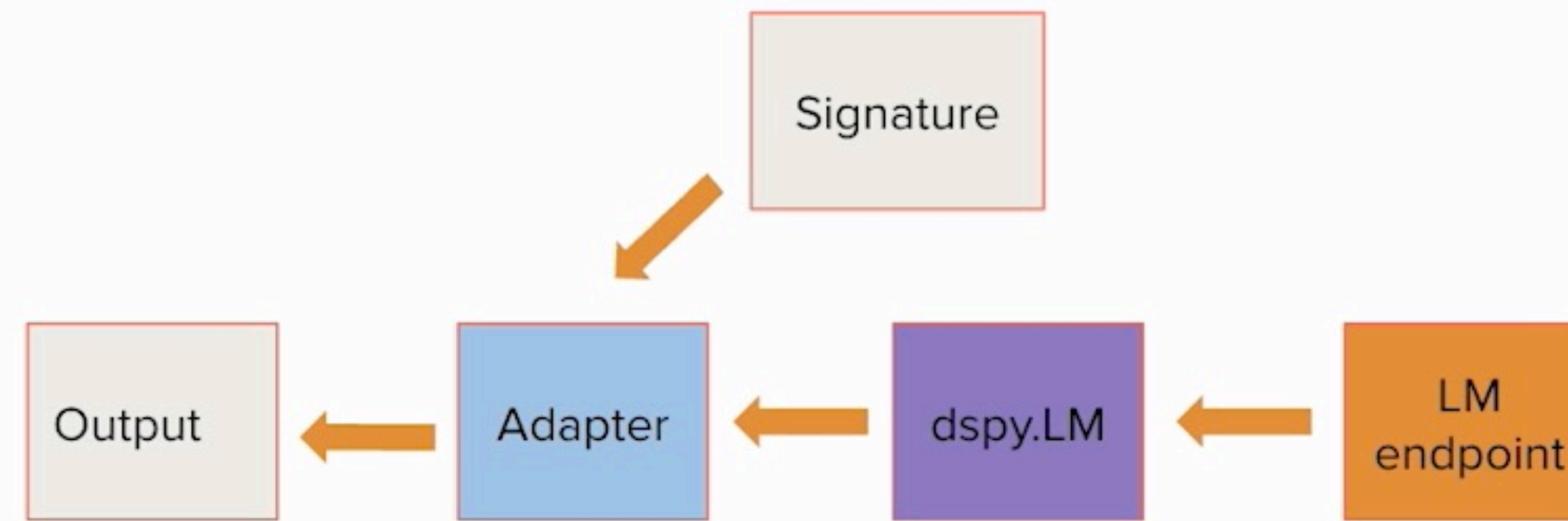
What is the meaning of life?

Respond with the corresponding output fields, starting with the field

[[## reasoning ##]], then [[## answer ##]], and then ending

with the marker for [[## completed ##]].

Behind the Scene - Adapter (Output flow)



The adapter takes in signature and LM response to parse the response into the user-specified fields.

Into the Adapter Output Flow

User message:

```
[[ ## question ## ]]  
What is the meaning of life?
```

Respond with the corresponding output fields, starting with the field `[[ ## reasoning ## ]]', then `[[## answer ##]]', and then ending with the marker for `[[## completed ##]]'.

Response:

```
[[ ## reasoning ## ]]  
The meaning of life is a philosophical question...  
  
[[ ## answer ## ]]  
The meaning of life is a subjective concept that varies...  
  
[[ ## completed ## ]]
```



```
Prediction(  
  reasoning="The meaning of life is a philosophical question...",  
  answer="The meaning of life is a subjective concept that  
varies..."
```